#11 Perform the range query collecting all points in the hyper-sphere with center `q`. In this implementation we assume the function does include the point `q` itself (as per the implementation in listing 9.11).
#12 If the number of points in `q`'s neighborhood (including `q` itself) is less than `minPoints`, then we don't need to do anything (`q` at this point is already marked as noise).
#13 Otherwise, add `q` to current cluster by setting its cluster index.
#14 Update the list of points to process by adding all of `q`'s neighbors that haven't yet been processed.
#15 Return the classification of points in the dataset (by means of their cluster indices).

You can also find a Python implementation on the book's repo, as well as a Jupyter notebook to experiment with the algorithm.

Obviously, for DBSCAN as well as for k-means, it is possible to use brute-force linear search to find each point's neighborhood; while for k-means, however, the speedup is nice but not vital (so much that usually k-means implementations never bother about this), for DBSCAN the performance gain would be dramatic: can you guess why? Before going further and reading the explanation, try to think about it for a minute.

So, the difference with DBSCAN is that the search extends on the whole dataset, while for k-means we only look for the closest among k centroid (and usually $k << n$).

If we have n points in the input dataset, then the difference, considering the running time of the whole algorithm, is